Software Architecture & Design Specification (SADS)

For: Online Quiz System

1. Introduction

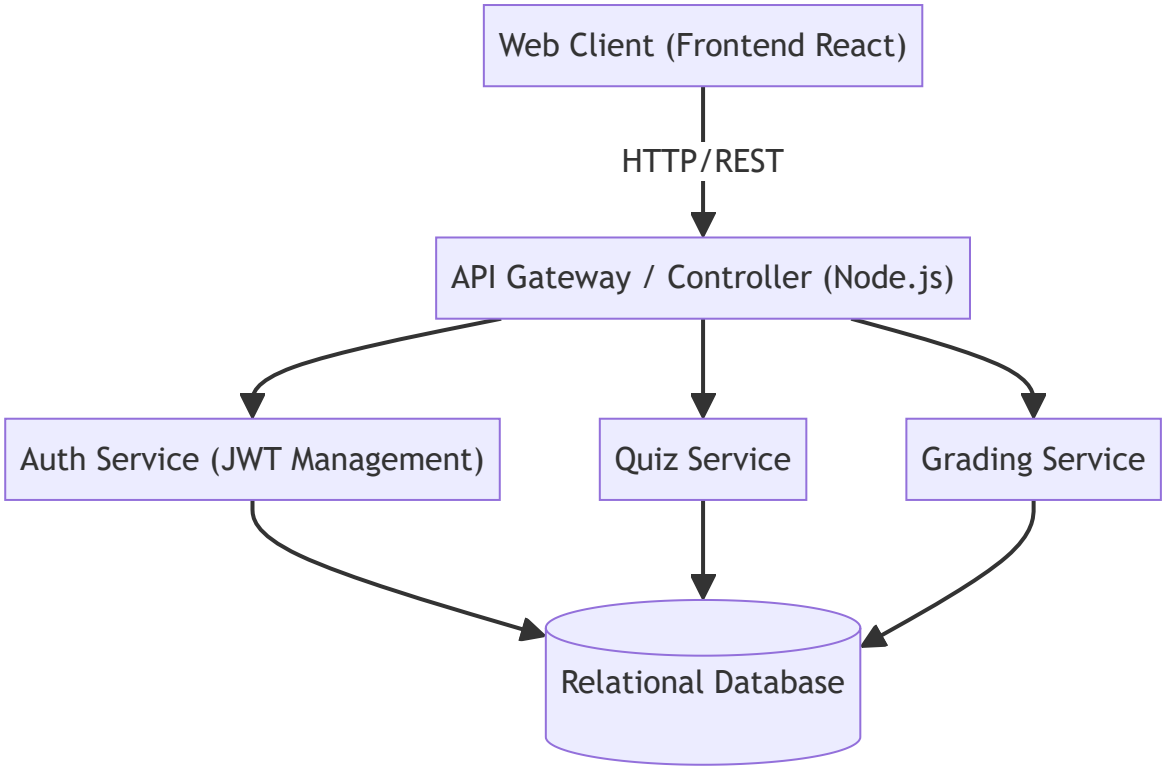
This Software Architecture and Design Specification provides a comprehensive structural and behavioral overview of the Online Quiz System. It adheres to standard IEEE format for describing software architectures.

2. Architecture

2.1 Architecture Pattern

The system is built upon a **3-Tier Architecture Pattern** and utilizes the **MVC (Model-View-Controller)** paradigm on the backend server. This cleanly separates concerns into the presentation layer (Client UI), the application logic layer (REST API Controllers), and the data access layer (Database Models), ensuring high modularity and long-term maintainability.

2.2 Component Diagram



2.3 Traceability to Requirements

- **Web Client:** Maps directly to FR1, FR2, FR3 by providing the graphical interface for these actions.
- **Auth Service:** Maps to FR1, SEC1, SEC2 by handling logins, password hashing, and role validation.
- **Quiz Service:** Maps to FR2, FR3 by handling the logic for creating, fetching, and persisting quizzes.
- **Grading Service:** Maps to FR4, NFR1 by efficiently calculating scores against answer keys in real-time.

2.4 Security Architecture

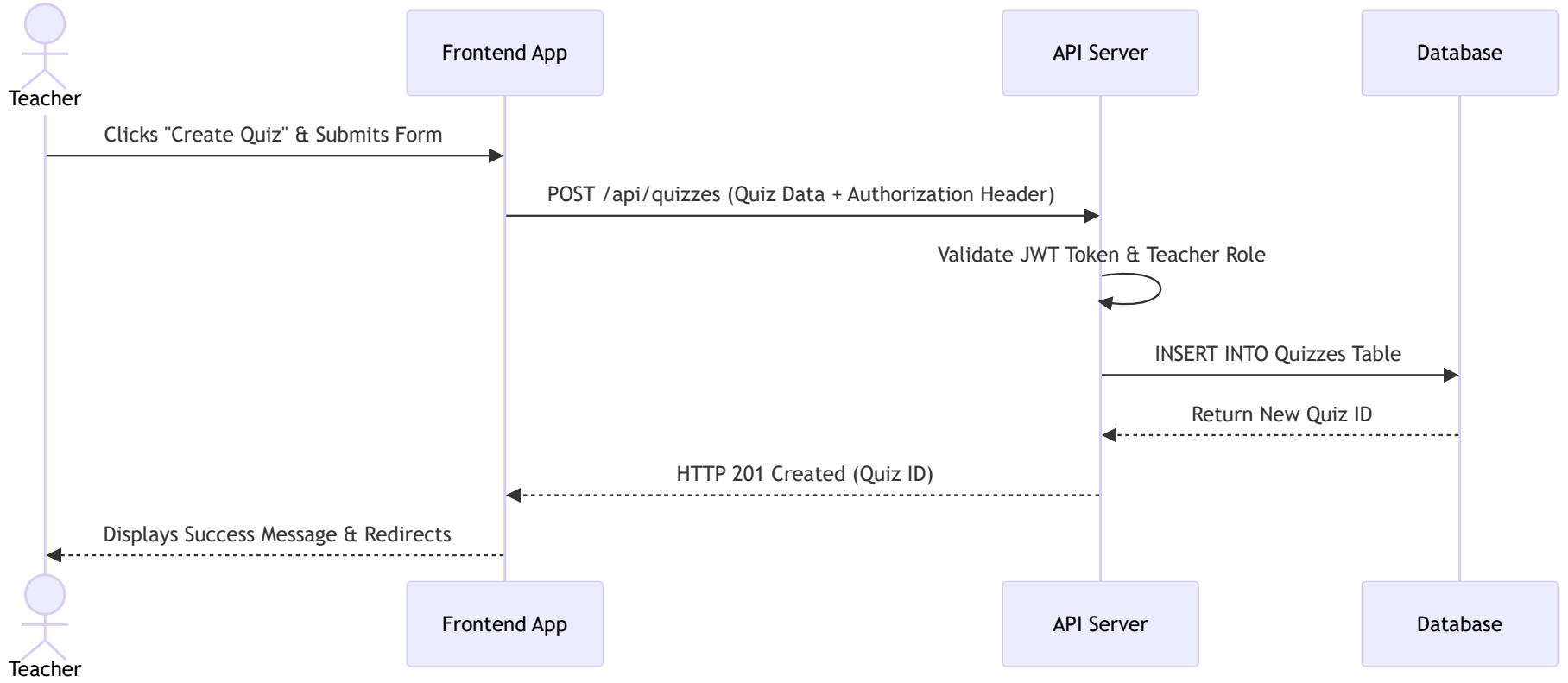
- **Authentication:** The system issues cryptographically signed JSON Web Tokens (JWT) upon successful login. Tokens use short expiration lifespans for security.

- **Authorization:** Custom middleware acts as a gatekeeper, verifying user roles embedded directly within the JWT before granting access to sensitive API routes.
- **Data Protection:** Passwords are mathematically hashed via bcrypt. All network communication is secured over TLS/SSL (HTTPS). SQL queries heavily utilize parameterized statements (via an ORM) to prevent SQL injection vulnerabilities.

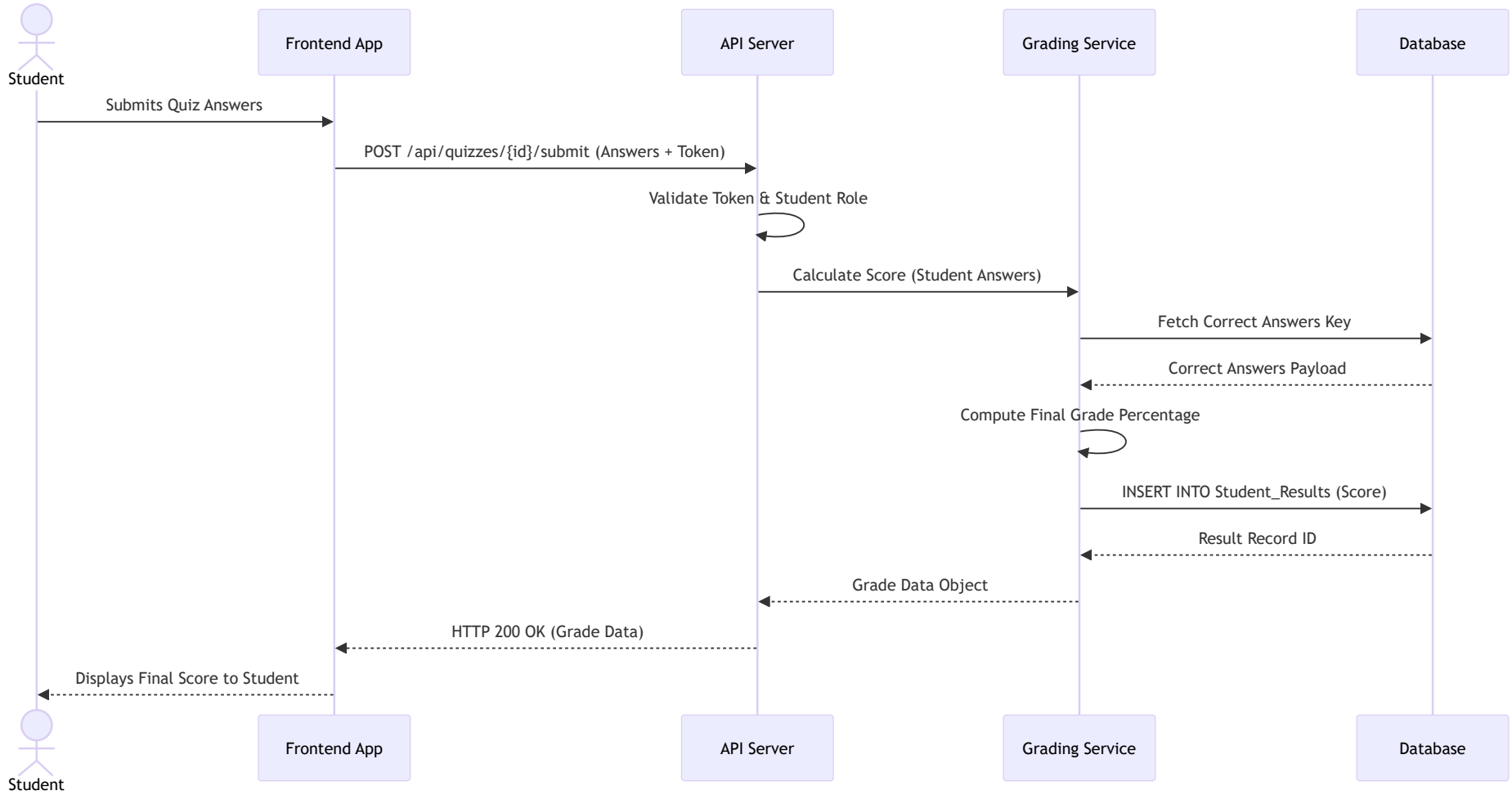
3. Design

3.1 UML Sequence Diagrams

Sequence Diagram 1: Quiz Creation (Teacher Workflow)



Sequence Diagram 2: Quiz Taking & Auto-Grading (Student Workflow)



3.2 API Design

The system exposes standardized RESTful endpoints that communicate exclusively via JSON:

- `POST /api/auth/login` : Authenticates user credentials and returns a signed JWT.
- `POST /api/quizzes` : Creates a new quiz payload (Strictly requires Teacher/Admin Authorization).
- `GET /api/quizzes/{id}` : Retrieves basic quiz details and questions for the student to answer.
- `POST /api/quizzes/{id}/submit` : Submits student answers for immediate grading (Strictly requires Student Authorization).

3.3 Error Handling

All API endpoints follow a unified error response structure utilizing standard HTTP status codes:

- **400 Bad Request:** Missing or invalid fields detected in the request body.
- **401 Unauthorized:** Missing, expired, or invalid JWT token.
- **403 Forbidden:** User is authenticated but lacks required role privileges (e.g., Student trying to create a quiz).
- **404 Not Found:** Requested resource (e.g., a specific Quiz ID) does not exist in the database.
- **500 Internal Server Error:** Unhandled runtime exception or catastrophic database failure.

Standard Error JSON Response Structure:

```
{
  "success": false,
  "error": {
    "code": 403,
    "message": "Access denied. Teacher privileges required to perform this action."
  }
}
```